# CHAPTER 1
# INTRODUCTION

## 1.1 PROJECT INTRODUCTION

The growth of internet and web technologies has influenced how people choose restaurants and order food. Many customers now prefer to check a menu, place an order, or reserve a table before visiting a restaurant. When these services are handled manually or through separate channels, it can be harder for customers to get timely information and for restaurant staff to manage requests efficiently. A web-based system can bring these activities together and make them easier to access. Yāzhi is a restaurant website designed to support two main services: online food ordering and table reservations. It also introduces the restaurant and its dining experience through Home, About, Menu, and Contact pages. The website uses a Tamil heritage visual identity, reflected in its branding, imagery, and design elements.Customers can register for an account and log in to access ordering and reservation features. They can browse food items by category, view details, and add dishes to a cart. At checkout, they provide relevant contact or address information, include an optional note, and select from the payment options presented by the application. Customers can later visit the My Orders page to review their orders and see the status assigned to each one.The reservation process allows customers to enter the date, time, and number of guests for their visit. They can then view the restaurant's interactive seating layout and choose from tables that meet the requested guest capacity and are available for that time. The system checks existing pending and approved bookings for time overlaps on each table. A reservation request is initially recorded as pending, allowing restaurant staff to review it before approval. Customers can check their reservation status and cancel eligible bookings.

The application also provides a separate admin panel for restaurant staff. Through it, administrators can manage food categories and menu items, update table information, review booking requests, and manage customer orders. They can approve or reject reservations and update order statuses as an order progresses. Server-side authentication and role checks protect administrative functions from access by regular customer accounts.

Yāzhi is implemented using a client-server structure. The frontend is built with React and Vite, with Tailwind CSS used for styling. The backend uses Node.js and Express.js to handle application requests through REST API routes. MongoDB, accessed through Mongoose, stores information related to users, categories, menu items, tables, bookings, and orders. JWT-based authentication is used to identify logged-in users and control access to protected functions. The project also includes optional Cloudinary image uploads and Nodemailer email notifications, which require service configuration.

By combining customer-facing services with restaurant administration, Yāzhi provides a single place to manage the main stages of ordering and reservation. Customers can use the website to explore the menu, submit orders, and request tables, while administrators can maintain restaurant information and respond to those requests through the management panel.

## 1.2 DOMAIN BACKGROUND & PROBLEM STATEMENT

**Domain Background**

The restaurant domain involves several connected activities, including presenting menu items, receiving food orders, arranging table reservations, and keeping customers informed about their requests. As customers increasingly use websites to access restaurant services, a digital platform can make these tasks available in one place.

Yāzhi is a web-based restaurant system that combines online food ordering and table reservations. Customers can browse menu categories and dishes, add items to a cart, submit an order, and view its status. They can also request a table by providing the date, time, and number of guests, then selecting an available table from an interactive seating layout. The system checks table capacity and existing pending or approved bookings when showing availability.

Restaurant staff use the admin panel to manage menu categories, food items, tables, reservations, and orders. Booking requests can be reviewed and approved or rejected, and order statuses can be updated. This gives customers and administrators access to related restaurant services through a shared application.

### Problem Statement

When menu information, food orders, and table reservations are handled through separate or manual processes, customers may need to contact the restaurant through different channels to complete routine tasks. Restaurant staff may also find it harder to keep menu details current, track order progress, review reservation requests, and check whether a table is already booked for a particular time.

The project addresses these needs by providing a centralized web application for food ordering and table reservations, together with administrative tools for managing the related information. It allows customers to browse the menu, submit orders, and request tables online. It also helps administrators maintain menu and table records, review bookings, and update orders. The system's availability check considers table capacity and overlapping pending or approved reservations, helping staff and customers make reservation decisions using current booking data.

## 1.3 OBJECTIVES AND MOTIVATION

### Objectives

The main objective of Yāzhi is to provide a web-based platform that supports restaurant food ordering and table reservations, along with administrative tools to manage these activities.

The specific objectives are:

- To let customers create accounts, log in, and manage access to their accounts.
- To provide an online menu where customers can browse food categories and view dish details.
- To allow customers to add dishes to a cart, place orders, and check order status.
- To provide an online table reservation process based on the date, time, and number of guests.
- To display restaurant tables through an interactive seating layout and check availability using table capacity and existing bookings.
- To allow customers to view their bookings and cancel eligible reservations.

- To provide administrators with tools to manage menu categories, food items, tables, reservations, and orders.
- To protect administrative functions through authenticated, role-based access.

**Motivation**

The project was motivated by the need to make common restaurant services accessible through one website. Customers may want to check menu details, place an order, or request a table without having to handle each task separately. A digital platform can make these services easier to access and give customers a way to review their orders and reservation status.

Restaurant staff also need a practical way to keep menu and table information organized and respond to customer requests. Yāzhi brings these functions into an admin panel, where staff can manage menu items and categories, review bookings, update order statuses, and maintain table details. The reservation availability check is included to help avoid assigning a table to overlapping bookings.

By combining customer services with restaurant management features, the project aims to provide a more organized experience for both customers and administrators. The project also aims to make the reservation process clearer for customers. Instead of submitting a request without knowing the seating arrangement, customers can view the restaurant's interactive table layout and select a table that matches their group size. Availability is checked against existing pending and approved reservations for the selected time, so the choice is based on booking information stored in the system.Another motivation for developing Yāzhi was to give restaurant administrators a central place to maintain important information. Menu categories, dishes, table details, bookings, and orders can be managed through the admin panel. This helps keep the information shown to customers connected with the records used by staff to process requests.

The project also provides an opportunity to apply full-stack development concepts in a practical setting. Its React-based frontend communicates with an Express backend through API routes, while MongoDB stores the application data. Authentication and role checks are used to control access to customer and administrative functions. Together, these parts demonstrate how a web application can connect user-facing services with the data and management processes behind them.

# CHAPTER 2

# WORKING ENVIRONMENT

The Restaurant Management System is developed to run on a standard desktop or laptop computer using a web browser, Node.js, and npm. Its frontend is built with React and Vite and styled using Tailwind CSS, while the backend uses Node.js and Express.js to handle application requests. MongoDB, accessed through Mongoose, stores user, menu, order, table, and reservation data. The frontend and backend run as separate services during development and communicate through REST APIs. Cloudinary for menu image uploads and Nodemailer for email notifications are optional services that require additional configuration.

## 2.1 HARDWARE REQUIREMENT

Yāzhi is a web-based application and can be developed and used on a standard desktop or laptop computer. The hardware should be capable of running a modern web browser, Node.js, the frontend development server, and the backend server. Since application data is stored in MongoDB, the computer also needs access to a local MongoDB installation or a network connection to a hosted MongoDB database. The project does not process unusually large datasets or require specialised computing hardware.

**TABLE 2.1.1 HARDWARE REQUIREMENTS**

| Hardware Component | Minimum Specification | Recommended Specification |
|---|---|---|
| Processor (CPU) | Dual-core processor, 2.0 GHz or higher | Quad-core processor, 2.5 GHz or higher |
| System Memory (RAM) | 4 GB | 8 GB or more |
| Storage | 2 GB of available space for the project and required dependencies | SSD with at least 5 GB of available space |
| Display | 1280 × 720 resolution | 1920 × 1080 resolution |
| Network | Internet or local network access for database connectivity and package installation | Stable broadband connection, especially when using hosted services |

A computer meeting the minimum specifications should be able to run the frontend and backend for basic development and demonstration. Additional memory and a solid-state drive can improve performance when installing dependencies and running both services at the same time. A stable network connection is needed when the project uses a hosted MongoDB database or optional services such as Cloudinary image storage and email delivery. Customers accessing the application need a device with a modern web browser and a suitable network connection; they do not need to install the development tools.

## 2.2 SOFTWARE REQUIREMENT

The Restaurant Management System uses a set of frontend and backend technologies to provide food ordering, table reservations, and restaurant administration. The table below lists the technologies identified in the project files and their roles.

**TABLE 2.2.1 SOFTWARE REQUIREMENTS**

| **Software / Technology** | **Use in the Project** |
|---|---|
| HTML | Provides the basic structure of the web pages rendered by the React application. |
| CSS and Tailwind CSS | Style the interface and support responsive page layouts. Tailwind CSS is the styling framework used in the project. |
| JavaScript | Implements frontend interactions and backend application logic. |
| React | Builds the user interface, including customer pages, reservation screens, and admin pages. |
| Vite | Runs the frontend development server and builds the application for production. |
| Node.js | Provides the runtime environment for the backend application. |
| Express.js | Handles backend routes, HTTP requests, middleware, and API responses. |
| MongoDB | Stores application data, including users, menus, categories, tables, bookings, and orders. |
| Mongoose | Defines MongoDB models and supports database operations. |

| Software / Technology | Use in the Project |
|---|---|
| JWT | Supports authentication and access to protected customer and administrator functions. |
| Axios | Sends requests from the frontend to the backend API. |
| Cloudinary | Provides optional cloud storage for menu images when configured. |
| Nodemailer | Supports optional email notifications, including password reset and booking confirmation messages, when configured. |
| Visual Studio Code or another code editor | Can be used to edit and manage the project source files. The project files do not specify a required editor. |
| Google Chrome, Microsoft Edge, Mozilla Firefox, or another modern browser | Provides access to the customer website and admin panel. |
| Git and GitHub | Can be used for source-code version control and repository hosting. Their use is not specified in the project files. |

The application's frontend and backend are run as separate services during development. Node.js and npm are required to install dependencies and start these services. A MongoDB database connection is required for the application to store and retrieve its data. Cloudinary and Nodemailer are optional integrations and require their respective configuration to provide image uploads and email delivery.

## 2.3 SYSTEM SOFTWARE & TECHNICAL STACK

Yāzhi is developed as a full-stack web application using JavaScript across both the frontend and backend. This allows the client interface and server-side application logic to work together through API requests. The system is divided into a React frontend, which presents the website and admin panel, and a Node.js and Express backend, which processes requests and communicates with the database.

React was selected for building the user interface. The application is organised into pages and reusable components for functions such as menu browsing, cart management, checkout, table selection, bookings, and administration. Vite is used as the frontend development server and build

tool. Tailwind CSS provides the styling utilities used to define the layout, colours, and responsive presentation of the website.

Node.js provides the runtime for the server-side application, while Express.js handles API routes, middleware, and responses. The frontend communicates with the backend using Axios. The backend separates its functionality into routes, controllers, models, configuration, and middleware, making it possible to handle features such as authentication, menu management, orders, table availability, and reservations in distinct parts of the application.

MongoDB is used to store the system's data, and Mongoose defines the data models and supports database operations. The stored information includes users, menu categories, food items, tables, bookings, and orders. JWT-based authentication is used to identify logged-in users and protect relevant API routes. Role checks restrict administrative operations to accounts with the administrator role.

The project also includes optional integrations for Cloudinary and Nodemailer. Cloudinary can be used to store uploaded menu images, while Nodemailer can send password reset and booking confirmation emails when the required service settings are provided. These integrations depend on external configuration and are not necessary for describing the core React, Express, and MongoDB application.

# CHAPTER 3

# SYSTEM ANALYSIS

System analysis examines the requirements, feasibility, existing challenges, proposed solution, benefits, and scope of the Yāzhi Restaurant Management System. It provides the foundation for designing an application that brings food ordering, table reservations, and restaurant administration into one web-based platform. The analysis considers the needs of customers who want to browse menus, place orders, and request tables, as well as administrators who manage menu items, tables, bookings, and order statuses. It also considers how the system uses stored booking information to check table availability and how authenticated access protects administrative functions.

## 3.1 FEASIBILITY STUDY

A feasibility study was considered to assess whether the Yāzhi Restaurant Management System could be developed using the available technologies and resources. The project uses a common web development stack and separates the customer website, backend services, and database. Its feasibility is assessed in terms of technical, economic, operational, and schedule factors.

### 3.1.1 Technical Feasibility

Yāzhi is built with React, Vite, and Tailwind CSS for the frontend, and Node.js and Express.js for the backend. MongoDB, accessed through Mongoose, stores information such as user accounts, menu items, orders, tables, and reservations. These technologies support the functions implemented in the project, including account authentication, menu management, order processing, and table availability checks.

The system can be run on a standard computer with Node.js and access to a MongoDB database. Optional features such as Cloudinary image uploads and Nodemailer email notifications require external service configuration. The project does not require specialised hardware for its core functions; therefore, it is technically feasible for development and demonstration in a typical academic environment.

### 3.1.2 Economic Feasibility

The project uses widely available development technologies and does not require a paid licence for its core software stack. Development can be carried out on a standard computer, and the database can be hosted locally or through a cloud database service. Optional services such as Cloudinary and email delivery may have their own usage limits or costs, depending on the selected provider and plan.

The application brings ordering, reservations, and administrative management into one system. This can reduce reliance on separate manual processes for handling menu information, booking requests, and order updates. The project does not include a measured financial return or cost analysis, so a specific monetary benefit cannot be stated. Based on its technology requirements and implemented scope, it is economically feasible as an academic project.

### 3.1.3 Operational Feasibility

The application provides separate functions for customers and administrators. Customers can register, browse the menu, place orders, request tables, and view their order or booking status. Administrators use the admin panel to manage menu categories, food items, tables, bookings, and orders.

The reservation flow displays an interactive seating layout and checks table capacity and existing pending or approved bookings for time overlap. These features support the main tasks users need to complete within the system. The project is operationally feasible provided that administrators maintain current menu, table, and booking information and that the backend and database are available.

### 3.1.4 Schedule Feasibility

The system is divided into distinct frontend pages and components, backend routes and controllers, and database models. This structure supports implementation of its major functions as separate modules, including authentication, menu management, ordering, table reservations, and administration. The project files do not specify a development schedule or recorded milestones, so

completion within a particular timeframe cannot be verified. Its modular structure nevertheless makes it suitable for staged development and academic project work.

## 3.2 EXISTING SYSTEM

The project files do not describe a specific existing restaurant system or document the processes that were used before Yāzhi. In a manual or disconnected approach, customers may need to contact a restaurant separately to ask about menu items, place orders, or request a table. Staff may then record these requests and update menu, table, booking, and order information through separate methods. This can make it harder to maintain consistent records and provide customers with a convenient way to check request status.

Yāzhi addresses these common process challenges by bringing menu browsing, food ordering, table reservations, and administrative management together in one web application. Customers can submit orders and booking requests online, while administrators can manage menu and table records, review reservations, and update order statuses from the admin panel. The application also checks table capacity and overlapping pending or approved bookings when determining availability.

# 3.3 DRAWBACKS OF THE EXISTING SYSTEM

The project files do not document a specific system previously used by the restaurant. However, when food orders, table reservations, and menu records are handled through separate or manual processes, several difficulties may arise:

- **Requests handled through separate channels:** Customers may need to contact the restaurant separately to ask about menu items, place orders, or request a table.
- **Difficulty tracking order progress:** Without a shared record, customers may have limited access to the current status of an order, while staff may need to check updates manually.
- **Manual reservation coordination:** Staff may have to compare booking details to determine whether a table is available for a particular date and time. This can make it harder to identify overlapping reservations.

- **Scattered customer and restaurant records:** Menu details, table information, orders, and booking requests may be stored or updated separately, making them harder to review together.
- **Limited customer access to booking information:** Customers may need to contact the restaurant to confirm a reservation or ask whether it has been approved.
- **Repeated administrative effort:** Staff may need to update menu information, table details, booking decisions, and order statuses in different places.

Yāzhi addresses these potential limitations by providing a shared web application for menu browsing, ordering, reservations, and administrative management. Its backend checks table capacity and time overlap against existing pending or approved bookings, and customers can view their own order and booking information through the website.

## 3.4 PROPOSED SYSTEM

The proposed Yāzhi Restaurant Management System is a web application for online food ordering, table reservations, and restaurant administration. It provides customers with a website to browse menu categories and dishes, add items to a cart, submit orders, and view order status. Customers can also request a table by entering the date, time, and number of guests, then selecting a suitable table from an interactive seating layout.The system checks table capacity and compares the requested time with existing pending and approved reservations. This helps identify tables that are available for the selected time. Reservation requests are stored with a pending status until an administrator reviews them.

Customers can view their bookings and cancel eligible reservations.The admin panel provides functions for managing menu categories, menu items, tables, bookings, and orders. Administrators can review reservation requests, approve or reject them, manage table details, and update order statuses. Authentication and role checks protect customer and administrator functions.

The main features of the proposed system include:

- Customer registration, login, and password recovery.
- Menu browsing by category and viewing of individual menu items.

- Cart, checkout, order history, and order status viewing.
- Table reservation requests and an interactive seating layout.
- Backend checks for table capacity and overlapping active bookings.
- Customer access to booking details and eligible booking cancellation.
- Admin tools for managing categories, dishes, tables, bookings, and orders.
- Optional Cloudinary image uploads and Nodemailer email notifications when configured.

## 3.5 BENEFITS OF THE PROPOSED SYSTEM

The proposed system provides a shared platform for customers and restaurant administrators. Customers can browse the menu, place orders, request a table, and check their order or booking information through the website. This gives them a convenient way to access the services provided by the application.The interactive seating layout allows customers to view available tables based on party size and selected time.

Availability is determined using table capacity and existing pending or approved reservations. This helps reduce the chance of overlapping requests for the same table and time.The admin panel brings menu, table, reservation, and order management into one interface.

Administrators can update menu information, maintain table details, review reservation requests, and change order statuses. Role-based access control restricts administrative actions to authorized accounts.

The frontend, backend, and database are organized into separate parts, the project can be maintained and extended by updating the relevant application modules. Optional image and email services can also be configured to support menu image storage and customer notifications.

## 3.6 SCOPE OF THE PROJECT

The scope of Yāzhi covers the customer-facing website and the administrative functions implemented in the project. It includes:

- A restaurant website with Home, About, Menu, and Contact pages.
- Customer account registration, login, profile access, and password reset.
- Menu categories and food items with descriptions, prices, availability, and images.
- Cart management, checkout, order submission, order history, and order status display.
- Checkout fields for customer and address details, optional notes, and payment method selection.
- An interactive restaurant seating layout with table details and capacities.
- Reservation requests using the selected date, time, and number of guests.
- Backend table availability checks that consider capacity and overlapping pending or approved reservations.
- Customer access to booking history and cancellation of eligible bookings.
- An admin panel for managing menu categories, menu items, tables, bookings, and orders.
- Administrative booking actions, including approving, rejecting, and completing reservations.
- Administrative updates to order status.
- JWT-based authentication and role-based protection for administrative API functions.
- Optional Cloudinary image storage and Nodemailer email features, which require external configuration.

# CHAPTER 4

# SYSTEM DESIGN

The architecture and data flow of the Yāzhi Restaurant Management System. The application follows a client-server structure with a React frontend, a Node.js and Express backend, and a MongoDB database. Customers and administrators interact with the frontend through a web browser. The frontend sends requests to the backend API, where routes and controllers handle functions such as authentication, menu browsing, orders, bookings, and table availability. Mongoose models connect these operations to the database, which stores user, menu, table, order, and reservation information. This structure separates the user interface, application logic, and data storage, supporting the customer website and administrative panel within one system.

## 4.1 SYSTEM ARCHITECTURE

Yāzhi follows a client-server architecture with three main layers: the Presentation Layer, the Application Layer, and the Data Layer. Each layer has a separate role in supporting customer services and restaurant administration.

### 1. Presentation Layer (React Frontend):

The frontend is built with React and Vite and styled with Tailwind CSS. It displays customer pages for browsing the menu, managing the cart, placing orders, and requesting table reservations. It also provides the admin interface for managing menu items, tables, bookings, and orders. The frontend sends requests to the backend using Axios.

### 2. Application Layer (Node.js and Express Backend):

The backend handles API requests through Express routes, controllers, and middleware. It processes authentication, menu operations, orders, reservations, table availability checks, and administrative actions. JWT authentication and role checks protect restricted functions.

## 3. Data Layer (MongoDB and Mongoose):

MongoDB stores application records, including users, menu categories, menu items, tables, bookings, and orders. Mongoose models define the structure of this data and are used by the backend to perform database operations. Optional integrations include Cloudinary for menu images and Nodemailer for email notifications when configured.

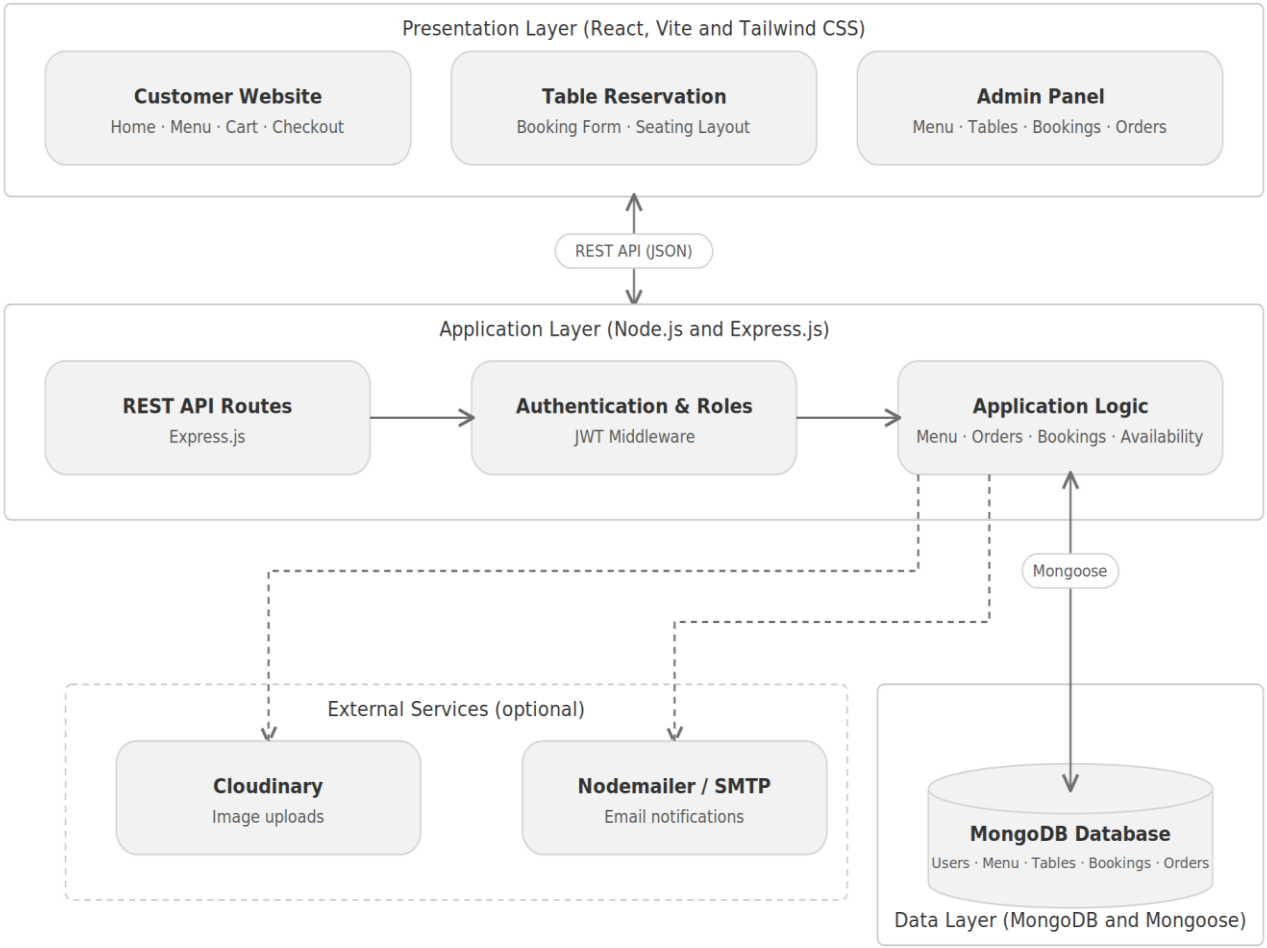



**Figure 4.1: Layered System Architecture of the Yāzhi Restaurant Ordering and Table Reservation System**

The React front end sends requests to an Express.js back end that handles authentication, business logic and bookings. Data is stored in MongoDB, with Cloudinary and Nodemailer as optional external services.

## 4.2 DATA FLOW DIAGRAMS (DFD)

Data Flow Diagrams show how information moves between customers, administrators, the Yāzhi system, and its database. In this project, data includes account details, menu information, cart contents, orders, table details, and reservation requests.

### 4.2.1 DFD Level 0 (Context-Level Diagram)

At Level 0, Yāzhi is represented as a single system. Customers provide account, order, and reservation information and receive menu details, order updates, and booking status. Administrators provide management actions and receive information about orders, bookings, menu items, and tables. The system reads and stores application records in the MongoDB database.

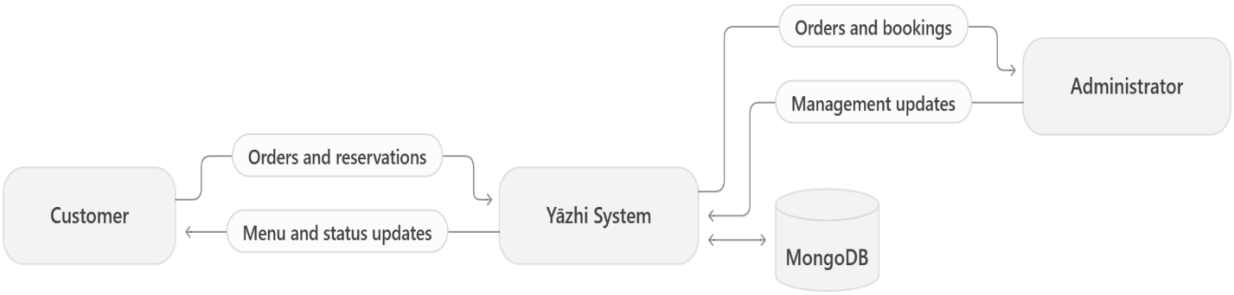



**Figure 4.2: Level 0 Context Data Flow Diagram**

Figure 4.2 presents the main interactions with the system. Customers use the website to browse menu items, submit orders, and request table reservations. Administrators use the admin panel to manage restaurant information and respond to customer requests. The system communicates with MongoDB to retrieve and store the data needed for these functions.

### 4.2.2 DFD Level 1 (Top-Level Process Breakdown)

The Level 1 DFD divides Yāzhi into its main processes:

- **Process 1.0 — Account and Authentication Management:** Registers users, verifies login details, supports password reset, and protects routes requiring authentication.
- **Process 2.0 — Menu and Category Management:** Retrieves menu items and categories for customers and allows administrators to add, update, or delete them.
- **Process 3.0 — Order Processing:** Maintains customer cart information, submits orders, provides order history, and allows administrators to update order status.
- **Process 4.0 — Table Reservation Management:** Receives reservation details, checks table capacity and availability, stores booking requests, and supports customer cancellation and administrator review.
- **Process 5.0 — Restaurant Administration:** Provides administrator functions for managing tables, bookings, menu information, and orders.

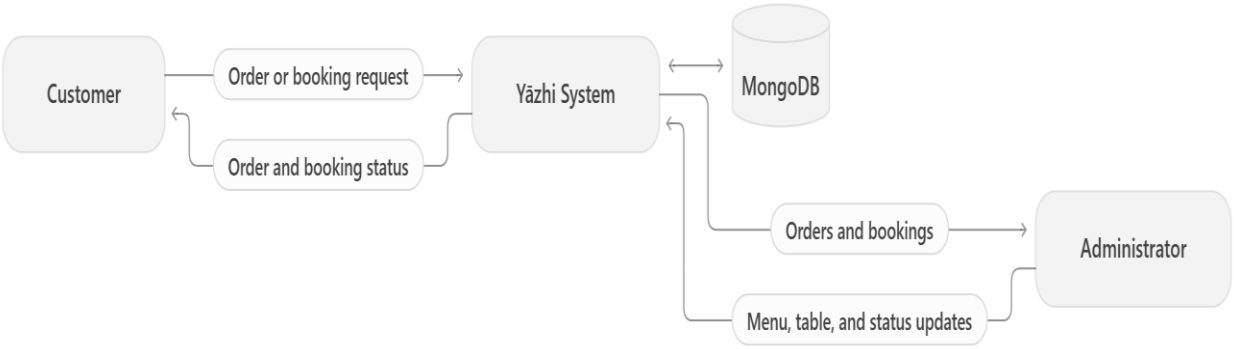



**Figure 4.3: Level 1 Top-Level Process Data Flow Diagram**

Figure 4.3 shows the main data flows within the application. Customer and administrator requests are handled by the relevant process, which reads or updates the corresponding MongoDB records. For example, order processing uses menu and user information to create an order, while reservation management uses table and booking records to determine availability and maintain booking status.

### 4.2.3 DFD Level 2 (Reservation Availability and Booking Flow)

The Level 2 DFD expands the reservation process. The customer provides the requested date, time, number of guests, and selected table. The system checks whether the table is active, whether its capacity is sufficient, and whether a pending or approved booking overlaps the requested time. If the request passes these checks, the system stores it with a pending status. An administrator can then approve or reject the request, and the customer can view the resulting status.

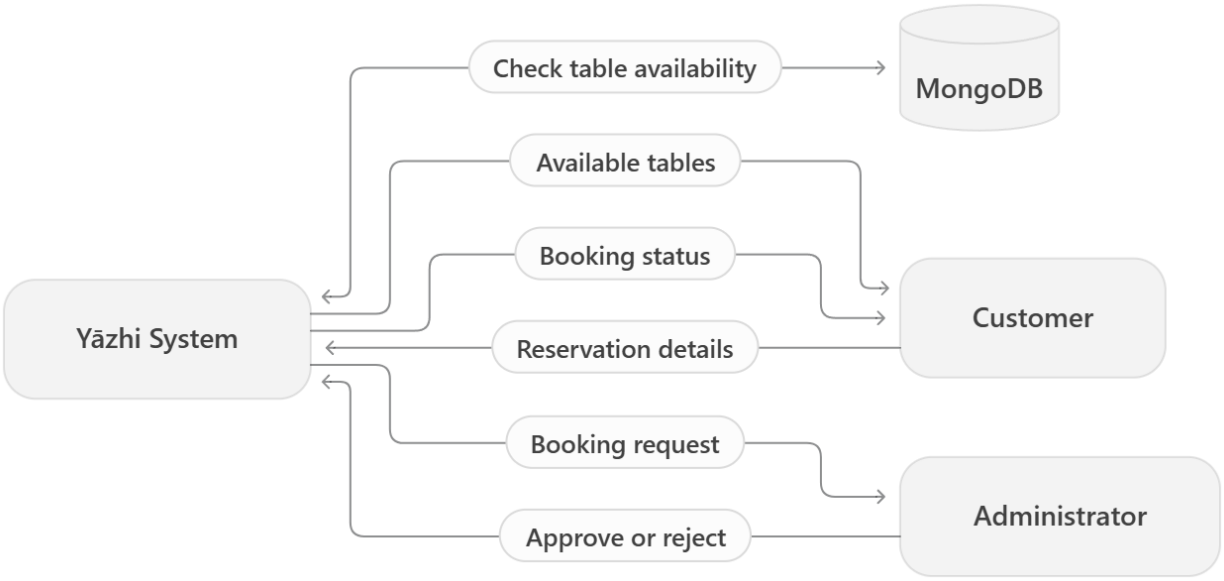



**Figure 4.4: Level 2 Reservation Data Flow Diagram**

Figure 4.4 details how a reservation moves through the system. Availability is based on active tables with sufficient capacity and the absence of an overlapping pending or approved booking for the requested time. A valid reservation is saved as pending for administrator review. The booking record is updated when an administrator makes a decision or when a customer cancels an eligible booking.

## 4.3 USE CASE DIAGRAM

The use case diagram shows how customers and administrators interact with the Yāzhi system. Customers can manage their accounts, browse the menu, place orders, and request or view table reservations. Administrators manage menu information, tables, bookings, and orders.

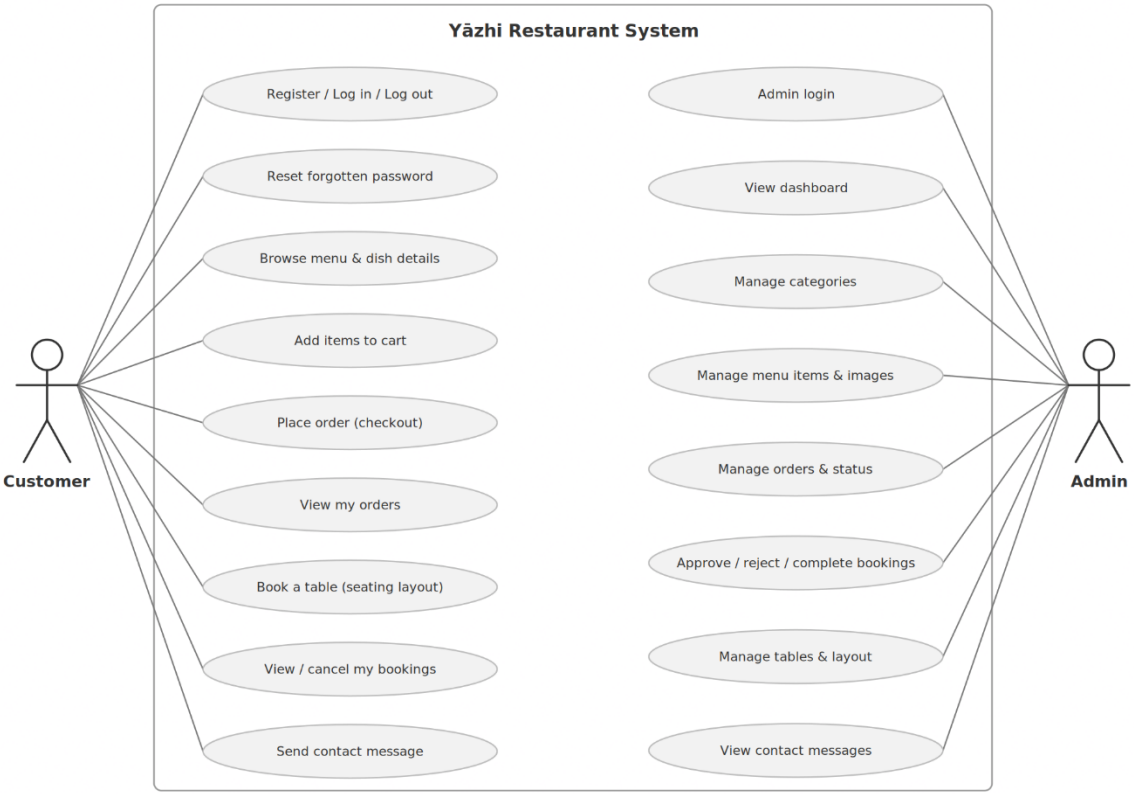



**Figure 4.5: UML Use Case Diagram for Enterprise Users and Analysts**

## 4.4 CLASS AND ACTIVITY DIAGRAMS

The class diagram represents the main data models used by the application. These models store information about users, menu categories, menu items, tables, bookings, orders, and customer contact messages.

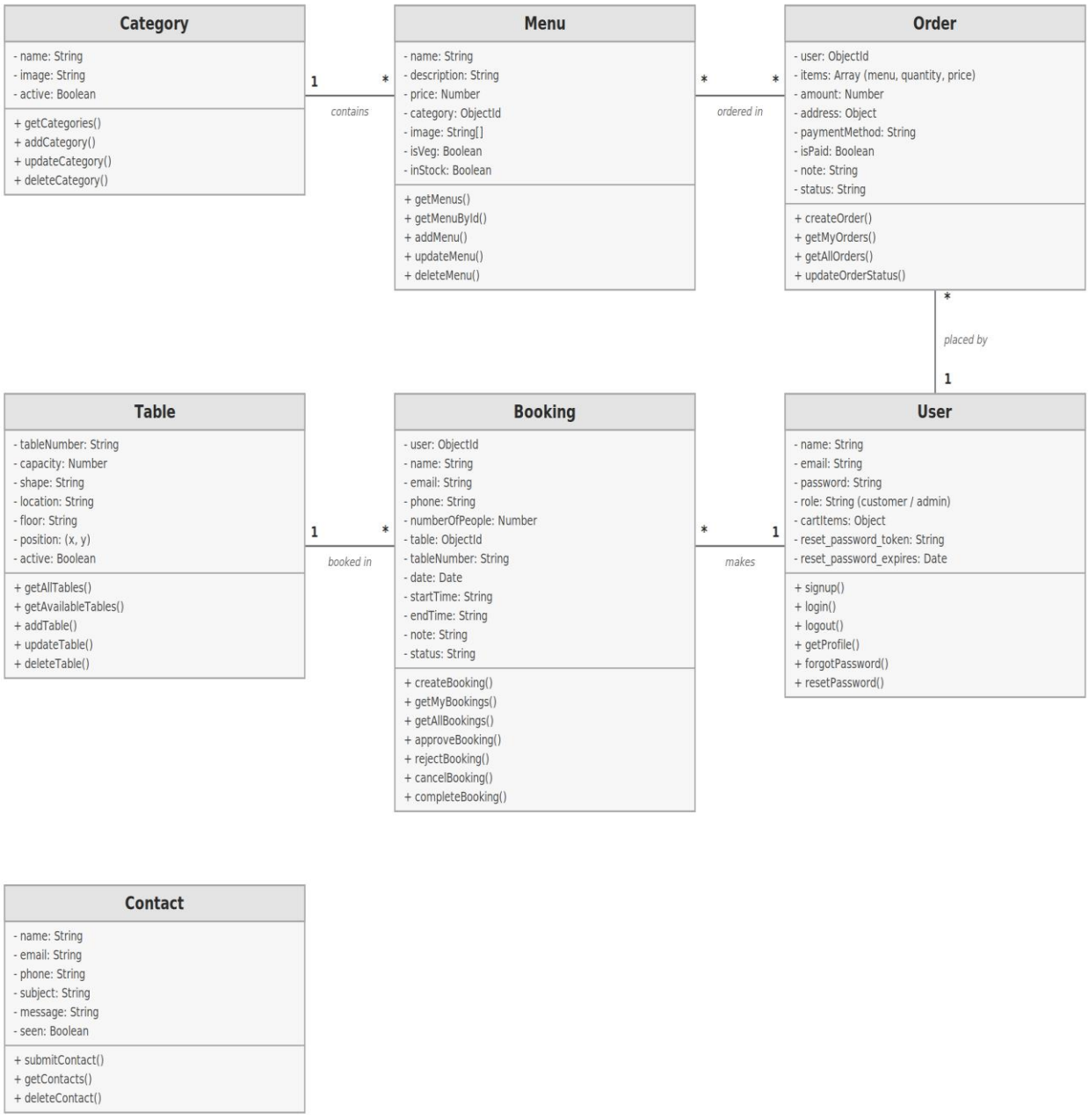



**Figure 4.6 Class Diagram for Restaurant Management System**

The activity diagram shows how a table booking moves between the Customer, the System and the Admin. The customer logs in, picks a date, time and number of guests, then selects a table on the seating layout. The system checks whether that table is free for the time slot. If it is, the booking is saved as Pending and a confirmation email is sent. If not, the customer picks another table. The admin then reviews the request and either approves or rejects it.

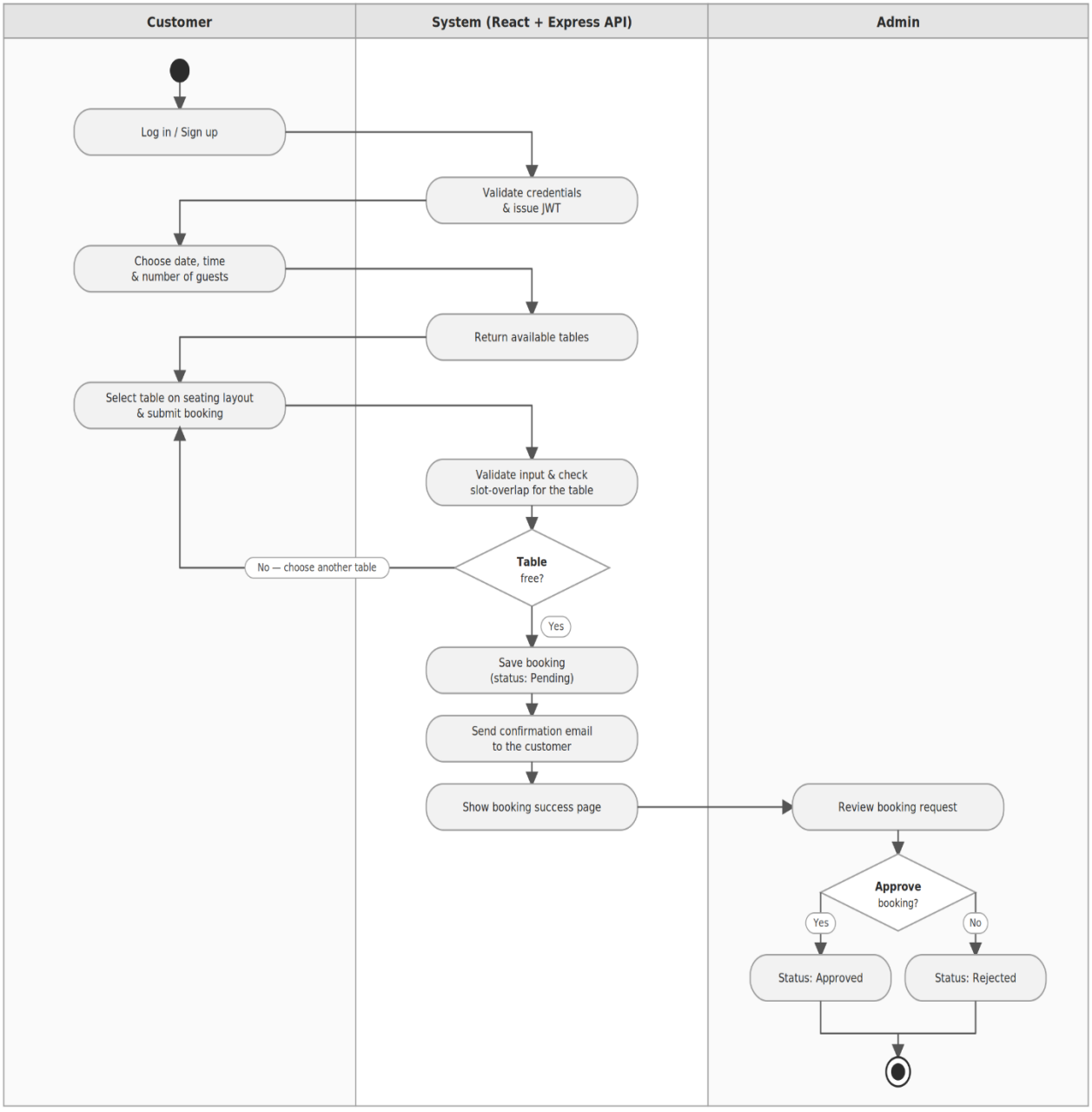



**Figure 4.7 Activity Diagram of the Table Booking Process**

## 4.5 DESIGN SYSTEM MODULES

The Yāzhi system is organized into customer service modules and administration modules. Each module handles a specific part of restaurant operations and communicates with the backend to retrieve or update the required information.

| **Module** | **Purpose** | **Main Output** |
|---|---|---|
| User Authentication | Supports customer registration, login, profile access, and password reset. | Authenticated customer account |
| Menu and Categories | Displays food categories and menu items; allows administrators to manage them. | Menu listings and item details |
| Cart and Checkout | Lets customers manage cart quantities and submit orders with contact and address details. | Submitted customer order |
| Order Management | Shows customers their orders and allows administrators to update order statuses. | Order history and current status |
| Table Reservations | Collects reservation details and checks table capacity and booking overlap. | Available tables and reservation request |
| Booking Management | Lets customers view or cancel eligible bookings and administrators review requests. | Updated booking status |
| Table Management | Allows administrators to add, edit, or deactivate restaurant tables. | Updated table information |
| Contact | Collects messages from customers and allows administrators to review them. | Stored customer enquiry |

**Table 4.1: Design System Modules**

## 4.5 USER INTERFACE DESIGN WIREFRAMES

Yāzhi's user interface is built with React and styled using Tailwind CSS. It uses a Tamil heritage visual identity, with a palette and design elements such as brass, terracotta, banana leaf, and kolam motifs. The interface is organized into customer pages and a separate admin panel.

- **Customer navigation:** The website includes pages for Home, About, Menu, Contact, account access, ordering, and reservations.
- **Menu pages:** Customers can browse menu items and categories, view item details, and add items to the cart.
- **Cart and checkout:** Customers can review items, change quantities, and enter relevant contact and address details before submitting an order.
- **Reservation pages:** Customers provide a date, time, and guest count, then use the interactive seating layout to select an available table.
- **Customer account pages:** Customers can review order and booking information, check statuses, and cancel eligible bookings.
- **Admin panel:** Administrators access separate screens to manage categories, menu items, orders, bookings, and tables.
- **Reusable interface elements:** Shared components such as the navigation bar, footer, menu cards, table cards, loading indicators, and booking status displays help keep the pages consistent.
- **Responsive styling:** Tailwind CSS classes are used to style page layouts and support presentation across different screen sizes.

# CHAPTER 5
# PROJECT DESCRIPTION

## 5.1 OBJECTIVE

The main objective of the Yāzhi Restaurant Management System is to develop a web application that supports food ordering, table reservations, and restaurant administration. The project brings these functions together so customers can use the website to browse the menu, place orders, request tables, and view their order or booking status, while administrators can manage the related information.

The specific objectives are:

- To provide customer registration, login, profile access, and password reset functionality.
- To let customers browse food categories and menu items, manage a cart, and submit orders.
- To allow customers to request a table by specifying a date, time, and number of guests.
- To display tables through an interactive seating layout and check availability using table capacity and existing bookings.
- To provide customers with access to their order history and booking details.
- To enable administrators to manage menu categories, menu items, tables, reservations, and orders.
- To protect administrative functions through authentication and role-based access checks.

These objectives connect the technical implementation with the main needs of a restaurant website. The customer features support ordering and reservation tasks, while the admin panel provides tools to maintain menu and table information and respond to customer requests.

## 5.2 MODULE DESCRIPTION

The Yāzhi Restaurant platform is organised into six functional modules. Customers use the public website, the account pages and the ordering and booking pages, while the admin works in a separate panel under /admin. All modules share the same Express REST API and MongoDB database. Access is controlled by JWT-based authentication with two roles, customer and admin, so each user only sees the pages and actions their role allows.

**Module 1: Customer Website and Menu**

This module is the public entry point and needs no login. It has a Home page with a hero banner, signature dishes, a "Moments Celebrated at Yāzhi" gallery, a "Meet Our Chefs" section and a customer reviews slider. It also has About and Contact pages. The Menu page shows dishes by category. Visitors can search by dish name and use a Veg-only filter, and they can open a Menu Details page for each dish with its images, description and price. Messages sent through the Contact form are saved for the admin.

**Module 2: User Authentication and Account**

This module handles registration and secure access. Users can sign up, log in and log out. A forgotten password is reset through a time-limited link sent by email. Passwords are hashed with bcryptjs. The login token (JWT) is checked on every protected request, and the admin has a separate login page.

**Module 3: Cart and Order Management (Customer)**

Logged-in customers add dishes to a cart that is saved to their account, so it is still there on their next visit. At checkout they enter their delivery details and an optional note, and choose Cash on Delivery or Pay at Restaurant. Each order has a status that moves from Pending through

Confirmed, Preparing, Ready and Out for Delivery to Delivered, and customers can follow it on the My Orders page.

**Module 4: Table Reservation**

This module lets customers book a table in three steps. They enter the number of guests, the date and the time, and past dates are rejected. They then choose a table on an interactive floor plan, where tables that are already booked or too small for the party are greyed out. The system checks for overlapping bookings, with a default reservation length of 90 minutes. The booking is saved as Pending, a confirmation email is sent to the customer's registered address, and a Booking Success page is shown. On the My Bookings page the customer can track the status and cancel a Pending or Approved booking.

**Module 5: Admin Dashboard**

The dashboard is the admin's landing page and gives a quick summary of the restaurant's activity. Its summary cards show Total Orders, Total Revenue, Menu Items, Categories, Total Reservations, Pending Reservations, Approved Reservations and Today's Reservations. A list of recent orders appears next to the cards. A sidebar leads to the other admin pages.

**Module 6: Admin Management**

This module covers day-to-day operations from the admin panel, in five areas:

- **Categories and menu items:** The admin adds, edits and deletes categories and menu items. Menu images are uploaded to Cloudinary, with up to four images per item.
- **Orders:** The admin updates the status of each order, from Pending to Delivered or Cancelled.
- **Bookings:** The admin searches reservations by name or table number, filters them by status, and then approves, rejects or completes them.

- **Tables:** The admin sets each table's number, capacity, shape, location, floor and position on the floor plan, and can mark a table inactive.
- **Contact messages:** The admin reads and deletes messages sent through the Contact form.

For the figures, take a screenshot of each page from your running app and put it above its caption. I can also write a table of booking statuses and what triggers each one, in the style of the hourly table in your reference. Tell me if you want it.

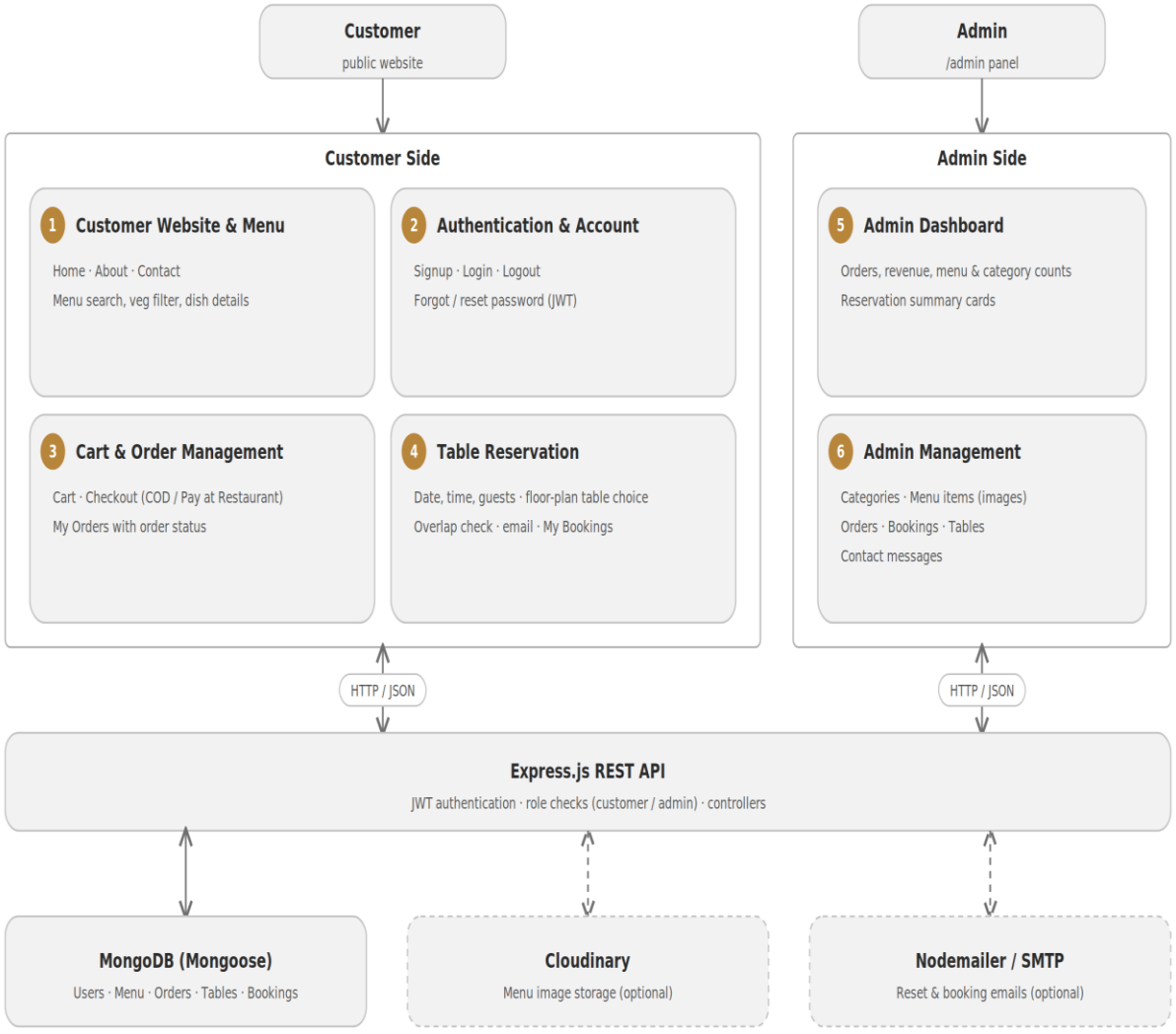



**Figure 5.1: Module Diagram of the Restaurant System**

The module diagram shows the Yāzhi system as six modules in two groups. The customer side covers the website and menu, authentication, cart and orders, and table reservation, while the admin side covers the dashboard and management pages. Both sides send requests to one Express.js REST API that checks JWT login and user roles, and the API reads and writes the MongoDB database, with Cloudinary for menu images and Nodemailer for emails as optional services.

## 5.3 IMPLEMENTATION AND ALGORITHMIC DETAILS

Yāzhi is implemented as a full-stack web application with a React frontend, an Express backend running on Node.js, and a MongoDB database accessed through Mongoose. The frontend provides customer and admin pages, while the backend exposes REST API routes for authentication, menus, orders, tables, bookings, categories, and contact messages.

### 1. Frontend Implementation

The frontend is built with React and Vite and styled using Tailwind CSS. React Router connects the different pages, including the menu, cart, checkout, bookings, orders, and admin panel. Shared components and application context support common elements and state such as the logged-in user and cart contents. Axios sends requests to the backend API.

### 2. Backend and Database Implementation

The backend uses Express routes to direct requests to the relevant controllers. Mongoose models define the stored data for users, categories, menu items, tables, bookings, orders, and contact messages. The controllers process requests and read or update records in MongoDB.Customer and administrator access is protected using JWT authentication middleware. Passwords are hashed using bcryptjs. Administrative API routes also check the user's role before allowing management actions.

### 3. Menu and Order Processing

Menu items are retrieved with their associated categories and displayed on the customer website. Customers can search dishes, filter by category or vegetarian status, and add items to their cart.

During checkout, the application sends the selected items, quantities, prices, total amount, address details, payment method, and optional note to the backend. The order is then stored in MongoDB and can be viewed by the customer and administrator.The checkout interface offers Cash on Delivery and Pay at Restaurant. The project does not include an integrated online payment gateway. Administrators can update order statuses, which are stored with the order record.

**4. Table Availability and Reservation Processing**

Customers enter the date, start time, and number of guests before selecting a table. The backend retrieves active tables with enough capacity and checks existing Pending and Approved bookings for the requested date. A booking is considered a conflict when its time interval overlaps the requested interval on the same table.

The reservation duration is set to 90 minutes by default and can be configured through the RESERVATION_DURATION_MINUTES environment setting. When a customer submits a request, the backend repeats the conflict check before saving the booking as Pending. Administrators can approve or reject pending bookings and mark approved bookings as completed. The system also checks for a conflict again when a booking is approved.

The booking flow is:

**Enter reservation details → Find suitable tables → Check table and time availability → Submit request → Save as Pending → Administrator reviews request → Display booking status**

**5. Optional Image and Email Services**

Menu image uploads use Multer to process uploaded files and Cloudinary to store them when the service is configured. Nodemailer can send password reset and reservation confirmation emails when email settings are provided. These integrations are optional and depend on external service configuration.

### 5.3.1 ALGORITHMIC DETAILS

The main algorithms in Yāzhi handle authentication, order storage, and reservation availability.

**1. Authentication Algorithm**

When a user registers, the backend validates the submitted details and hashes the password using bcryptjs before storing the account. During login, the submitted password is compared with the stored hash. On success, the backend creates a JWT. Protected API routes verify the token and load the user account; administrator routes also check that the account has the admin role.

**2. Table Availability Algorithm**

The availability process uses the requested date, start time, and guest count:

1. Retrieve active tables whose capacity meets or exceeds the guest count.
2. Retrieve Pending and Approved bookings for the selected date.
3. Calculate the end time using the configured reservation duration.
4. Compare the requested time interval with existing booking intervals on each table. Two intervals overlap when the requested start is earlier than the existing end and the existing start is earlier than the requested end.
5. Return tables that meet the capacity requirement and have no overlapping booking.

The backend repeats the conflict check when the customer submits a booking and when an administrator approves it.

**3. Order Creation Algorithm**

The frontend sends the selected menu items, quantities, prices, total amount, contact and address details, payment method, and optional note to the order API. The backend associates the order with the authenticated user and stores it in MongoDB. Administrators can later update the order’s status.

**4. Menu Filtering Algorithm**

The menu page retrieves menu items and categories from the backend. In the frontend, the displayed items are filtered using the selected category, dish-name search text, and vegetarian-only option.

The overall application flow is:

**Customer or administrator action → React page sends API request → Express route and controller process request → Mongoose reads or updates MongoDB → API response updates the interface**

## 5.4 MAINTENANCE

Maintenance is needed to keep Yāzhi operating correctly as its software dependencies, database records, and optional services change.

**1. Software and Functional Maintenance**

The frontend and backend depend on separate npm packages. Dependency updates should be reviewed for compatibility with React, Vite, Express, Mongoose, and the other packages in use. Changes to features such as authentication, ordering, table availability, or admin management should be checked against the related frontend pages, API routes, controllers, and models.

**2. Database Maintenance**

MongoDB stores user, category, menu, table, booking, order, and contact data. Database access and connection settings should be maintained, and backups should be kept to reduce the risk of losing records. Menu and table information should be kept current so customers see accurate items, prices, capacities, and table availability.

### 3. Reservation and Order Maintenance

Reservation duration is configurable through an environment setting, so the value should be reviewed if restaurant booking practices change. The availability and overlap rules should also be reviewed when modifying the reservation workflow. Administrators should keep booking decisions and order statuses up to date so customers can see the latest information available in the system.

### 4. Optional Service Maintenance

Cloudinary and Nodemailer require valid service credentials and configuration. If either integration is changed, its settings and related functionality should be checked. Without these services configured, menu image uploads or email delivery may not work, while the core ordering and reservation functions remain part of the application.

Future development could include integrating an online payment gateway, adding delivery tracking, or extending customer feedback features. These functions are not part of the current implementation.

# CHAPTER 6
# SYSTEM TESTING

## 6.1 TESTING DEFINITION

System testing is the process of evaluating a developed application to verify that its major functions work according to the specified requirements. It helps identify errors in authentication, data processing, booking logic, access control and output generation.

Testing covers the complete workflow. This begins with user registration and login, continues through menu browsing, cart and order placement and table reservation, and ends with email notifications and the admin management functions. The system is built with a React front end, an Express.js REST API and a MongoDB database. It includes a seeded floor plan of 10 tables, 7 menu categories and 8 API route groups: auth, tables, booking, menu, categories, orders, contact and cart. Testing checks whether these parts process data correctly and give users the right responses.

## 6.2 TESTING OBJECTIVE

The primary objective of testing is to verify that the Yāzhi Restaurant System works as intended and that its modules give correct, secure outputs.

The specific objectives are:

- To verify that user signup and login work correctly and return a valid JWT token.
- To check that duplicate emails and wrong passwords are rejected.
- To verify that a logged-in user can reserve a table and that the booking is saved with the correct status and time slot.
- To check that double booking of a table, and bookings that exceed the table's capacity, are prevented.
- To verify the forgot-password and reset-password flow, including token expiry and single use.
- To verify that admin-only routes can only be used by users with the admin role.

## 6.3 TYPES OF TESTING

The quality assurance process uses several testing methods to verify the functionality, performance, usability and reliability of the system.

### 1.Unit Testing

Unit testing verifies individual functions in the back end. Helper functions such as the time calculation that adds the 90-minute reservation duration, the time-to-minutes conversion and the booking-conflict check are tested to confirm they return the expected outputs.

### 2.Integration Testing

Integration testing verifies that the different parts of the system work together correctly. It examines the interaction between the React pages, the Axios requests, the Express routes, the JWT middleware, the Mongoose models and the MongoDB database. For example, submitting the booking form should create a booking record and trigger the confirmation email.

### 3.Performance And Load Testing

Performance testing examines how quickly the system responds to requests. It checks the response time of the menu and table listing APIs, the handling of image uploads of up to 5 MB, and the behaviour of the application when several users book tables at the same time.

### 4.Usability Testing

Usability testing examines whether the website is easy to understand and use. The evaluation covers the navigation bar, the menu search and veg filter, the interactive floor plan with its colour legend, the toast notifications, and the layout on desktop and mobile screens.

### 5.Regression Testing

Regression testing verifies that existing features still work correctly after changes or new additions. For example, changes to the booking module should not affect login, password reset or admin access control.

These testing methods verify the main functions of the system and identify areas for further improvement.

## 6.4 TEST CASES AND EXECUTION RESULTS

The following test cases cover the major modules of the Restaurant System.

**Test Case 1: User Registration and Login**

**Test Case ID:** TC-01

**Objective:**
To verify that a new user can register and log in with valid credentials.

**Input Conditions:**

- Send a signup request with a name, a new email and a password.
- Send a login request with the same email and password.

**Expected Results:**

- The signup should succeed and return a JWT token.
- The login should succeed and return a JWT token.
- The password should be stored in hashed form in the database.

**Observed Result:**

*Table 6.1: Test Results for User Registration and Login*

| **Parameter** | **Observed Result** |
|---|---|
| Endpoint | POST /api/auth/signup and POST /api/auth/login |
| Signup response | Success, token returned |
| Login response | Success, token returned |
| Password storage | Hashed (bcryptjs) |
| Status | PASS |

**Result: PASS**

**Test Case 2: Duplicate Email and Wrong Password**

**Test Case ID:** TC-02

**Objective:**
To verify that invalid registration and login attempts are rejected.

**Input Conditions:**

- Register again with an email that already exists.
- Log in with a correct email and a wrong password.

**Expected Results:**

- The duplicate registration should be rejected with an error message.
- The wrong-password login should be rejected with an error message.

**Observed Result:**

*Table 6.2: Test Results for Duplicate Email and Wrong Password*

| **Parameter** | **Observed Result** |
|---|---|
| Duplicate email signup | accepted, HTTP 400 |
| Wrong password login | accepted, HTTP 400 |
| Token issued | No |
| Status | PASS |

**Result: PASS**

**Test Case 3: Table Booking and Confirmation Email**

**Test Case ID:** TC-03

**Objective:**
To verify that a logged-in user can reserve a table and that the booking is saved correctly.

**Input Conditions:**

- Log in as a customer.
- Select an active table and send a booking request for 2 guests, for tomorrow's date at 18:00.

**Expected Results:**

- The booking should be created with the status Pending.
- The end time should be calculated automatically, 90 minutes after the start time.
- A confirmation email should be sent to the user's registered email address.
- If the email cannot be delivered, the booking should still be saved and the user should see a warning.

**Observed Result:**
*Table 6.3: Test Results for Table Booking and Confirmation Email*

| **Parameter** | **Observed Result** |
|---|---|
| Endpoint | POST /api/booking/create |
| Guests / start time | 2 guests / 18:00 |
| Booking status | Pending |
| End time | 19:30 (90-minute slot) |
| Confirmation email | Sent when SMTP is configured, warning shown otherwise |
| Status | PASS |

**Result: PASS**

# CHAPTER 7
# CONCLUSION

## 7.1 SUMMARY

The Restaurant Management System was developed as a web application for food ordering, table reservations, and restaurant administration. It brings customer services and administrative functions together, allowing customers to browse menu items, submit orders, request tables, and view their order and booking information.

The customer website includes account registration and login, menu categories and dish search, a cart, checkout, order history, and reservation pages. Customers can choose a table through an interactive seating layout. The system checks table capacity and overlapping Pending or Approved bookings when determining availability. Administrators can manage menu categories, dishes, tables, reservations, and order statuses through a separate admin panel.

The frontend uses React, Vite, and Tailwind CSS. The backend is built with Node.js and Express.js, and MongoDB with Mongoose stores user, menu, table, booking, and order data. JWT authentication protects account functions, while role checks restrict administrative API operations. Cloudinary image uploads and Nodemailer email notifications are included as optional integrations that require configuration.

Overall, Yāzhi provides a central web platform for customers to access restaurant services and for administrators to manage related records. The project demonstrates how a client-server application can connect a user interface, backend API, authentication, and database to support restaurant ordering and reservation workflows.

## 7.2 FUTURE ENHANCEMENTS

A future version of Yāzhi could include an online payment gateway so customers can pay securely during checkout. The current system provides Cash on Delivery and Pay at Restaurant options, and adding online payment would give customers another way to complete their orders.

Order tracking and notifications could also be introduced. Customers could receive updates as their orders progress, and messages could be sent when a reservation is approved, rejected, or changed. These updates would help customers follow their requests without repeatedly checking the website.

The system could be expanded with customer ratings and reviews, delivery management features, and reports for administrators. Reports on orders, revenue, popular menu items, and reservations could help restaurant staff review activity and plan their operations.

A dedicated mobile application could make Yāzhi's ordering and reservation services easier to access on smartphones. The application could use the same backend and database, allowing customer accounts, menu information, orders, and bookings to remain connected across the website and mobile platform.

# CHAPTER 8

# APPENDIX

## 8.1 SCREENSHOTS & UI WALKTHROUGH

This section presents screenshots of the Yāzhi Restaurant Management System. The figures show the customer website, menu and reservation pages, and administrative screens. Each description highlights the main interface elements visible in the corresponding screenshot.

### CUSTOMER FRONTEND

The home page opens with the Yāzhi logo and navigation links for Home, About, Menu, Book Table, and Contact. A large hero image, Tamil heritage branding, and buttons for exploring the menu or booking a table introduce the restaurant.

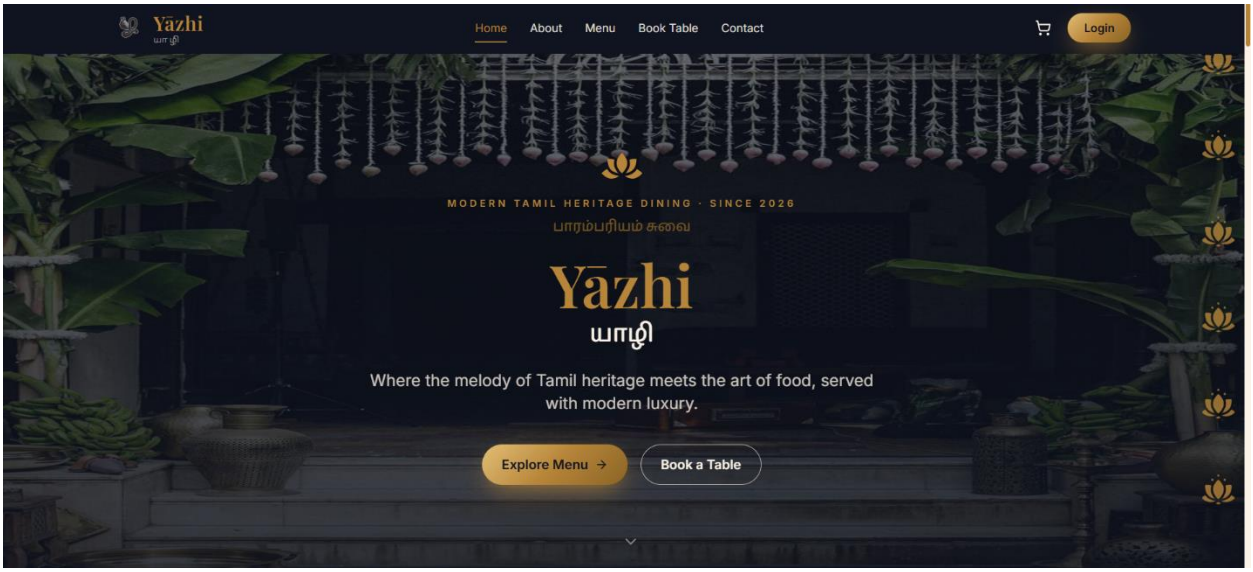
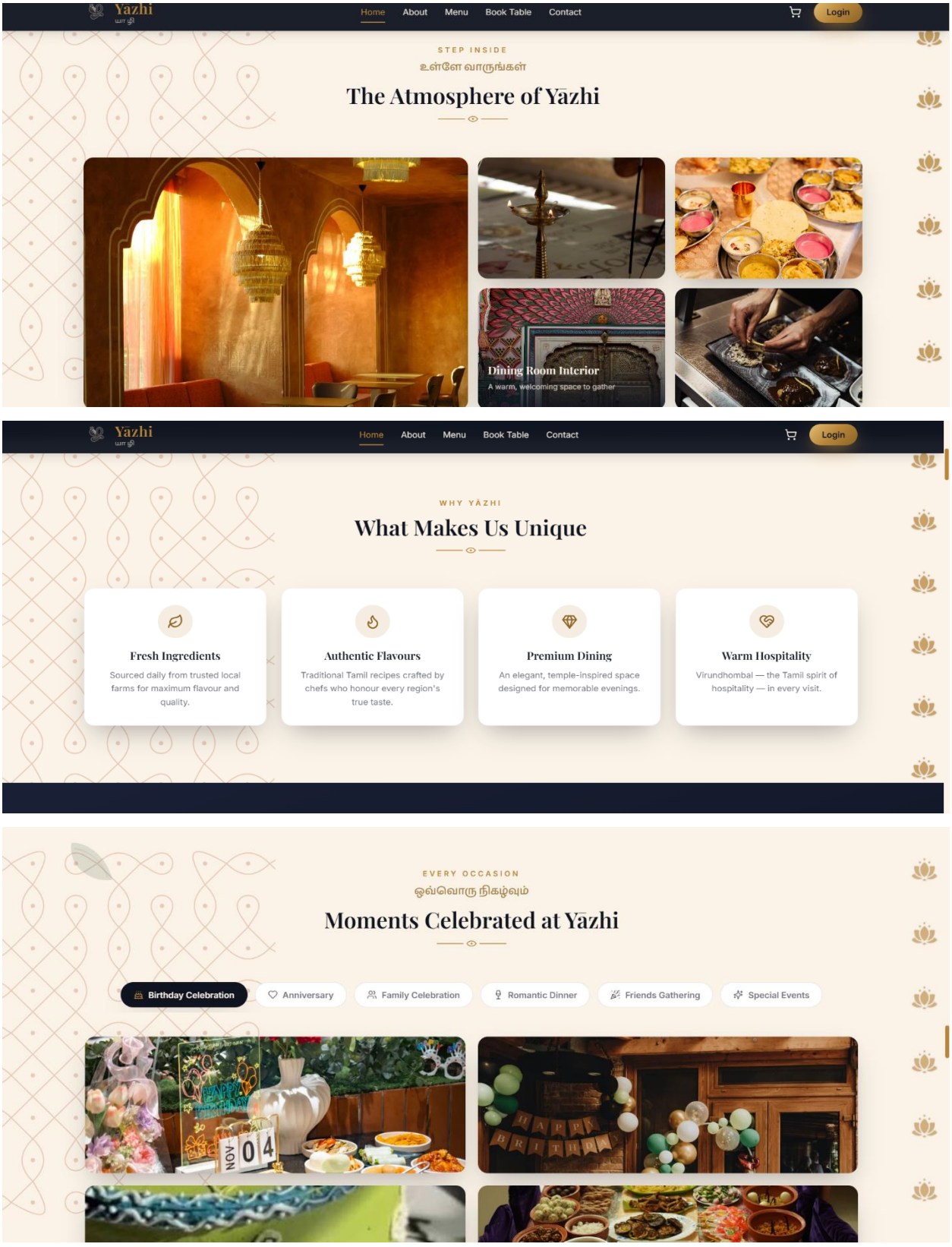
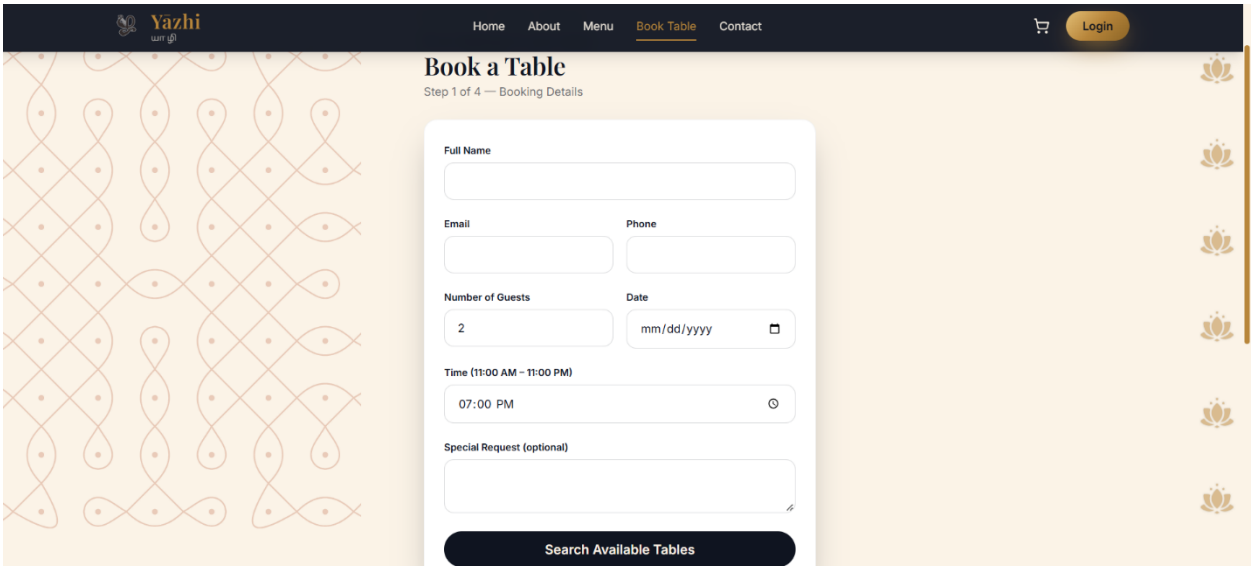


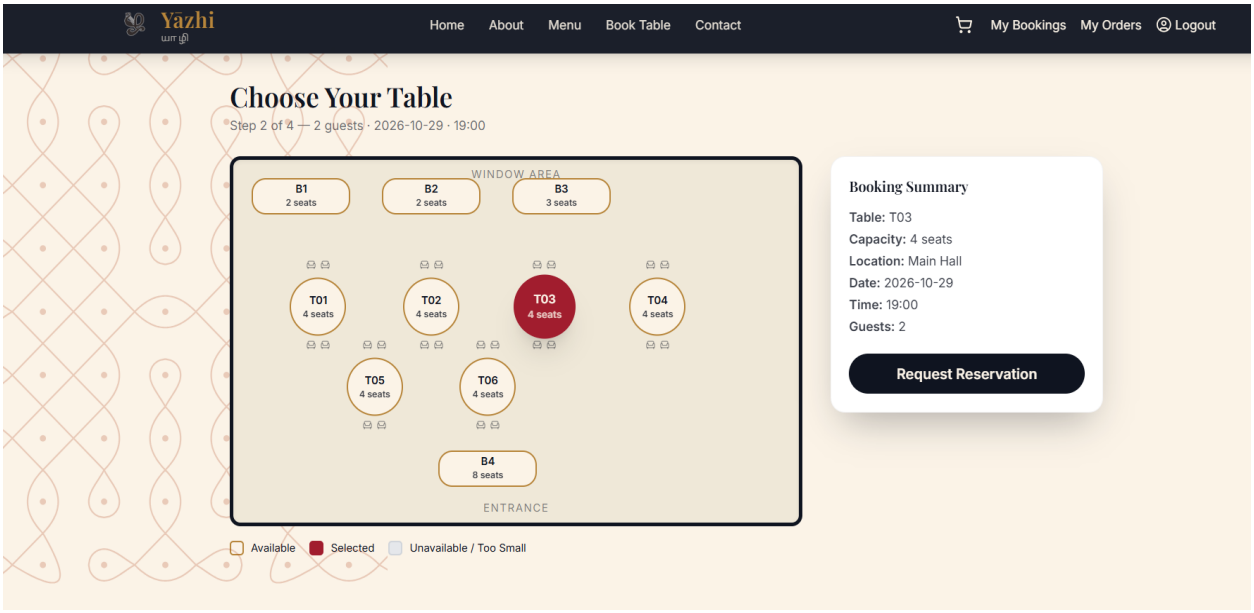

**Figure 8.1: Home Page Header**

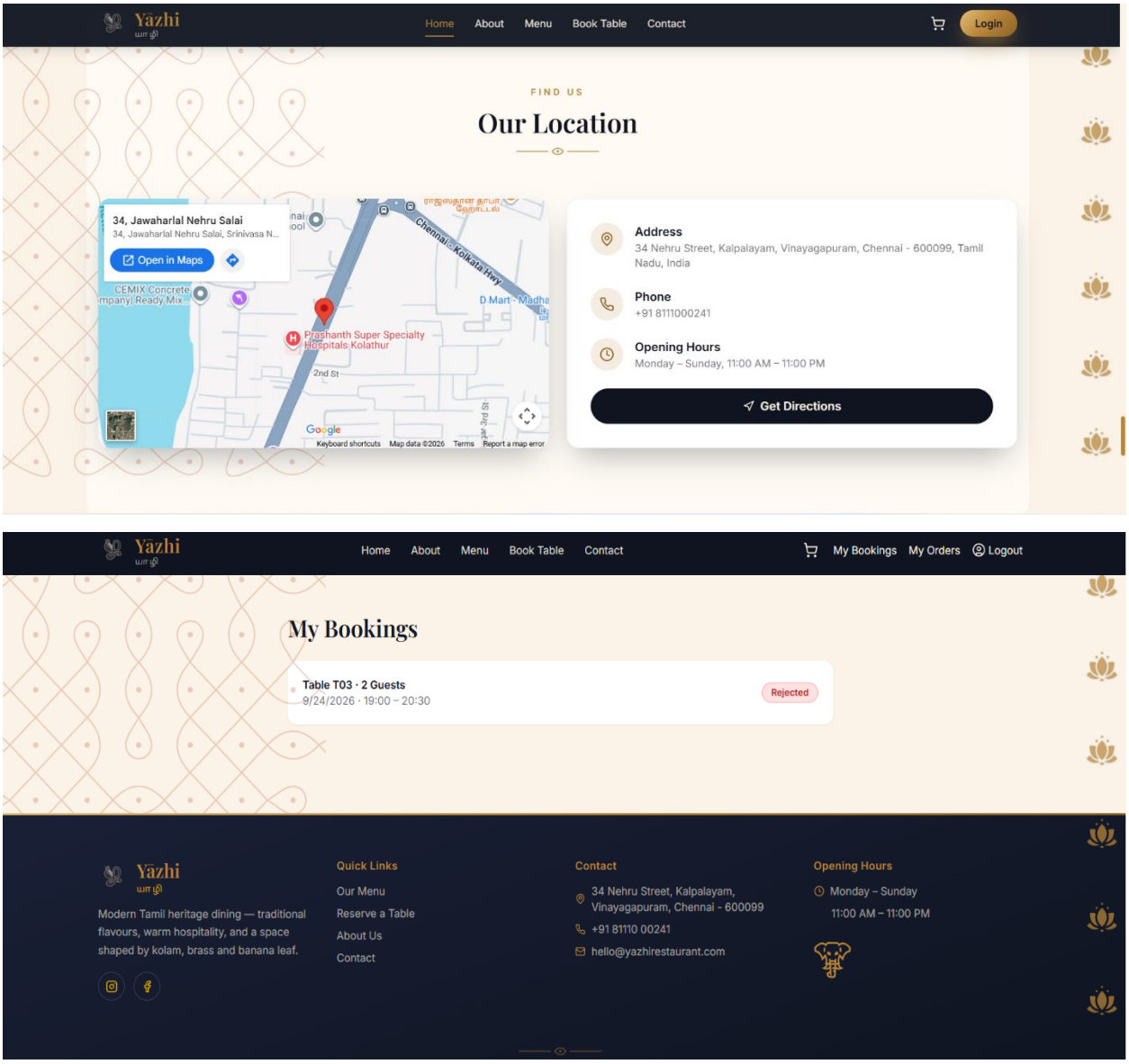
## FRONTEND SECTIONS (FIGURE 8.2)



## TABLE BOOKING SECTION (FIGURE 8.3)



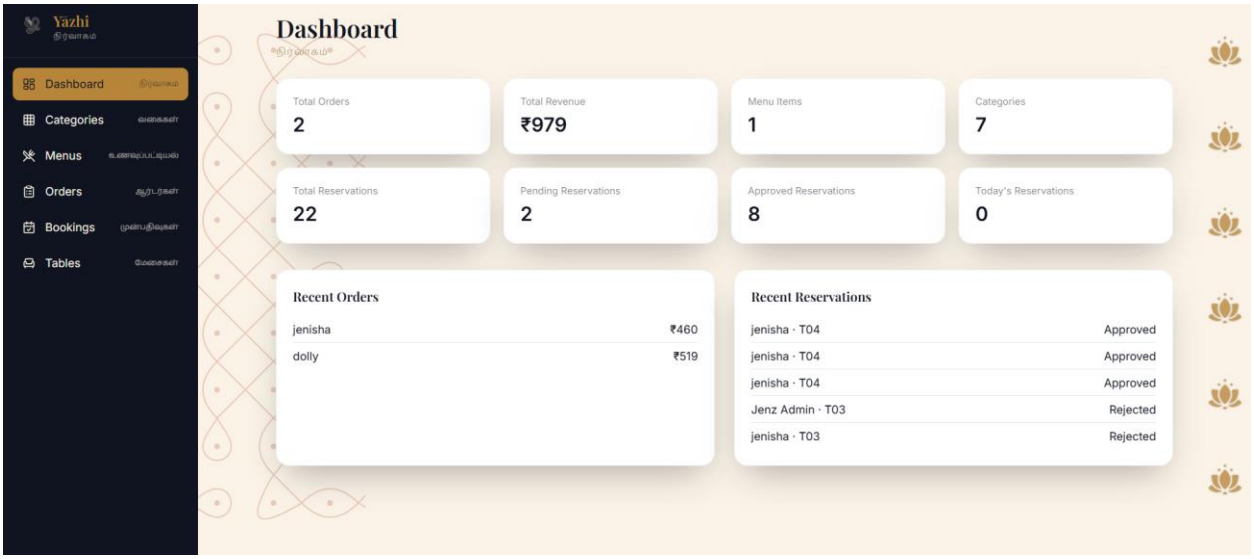


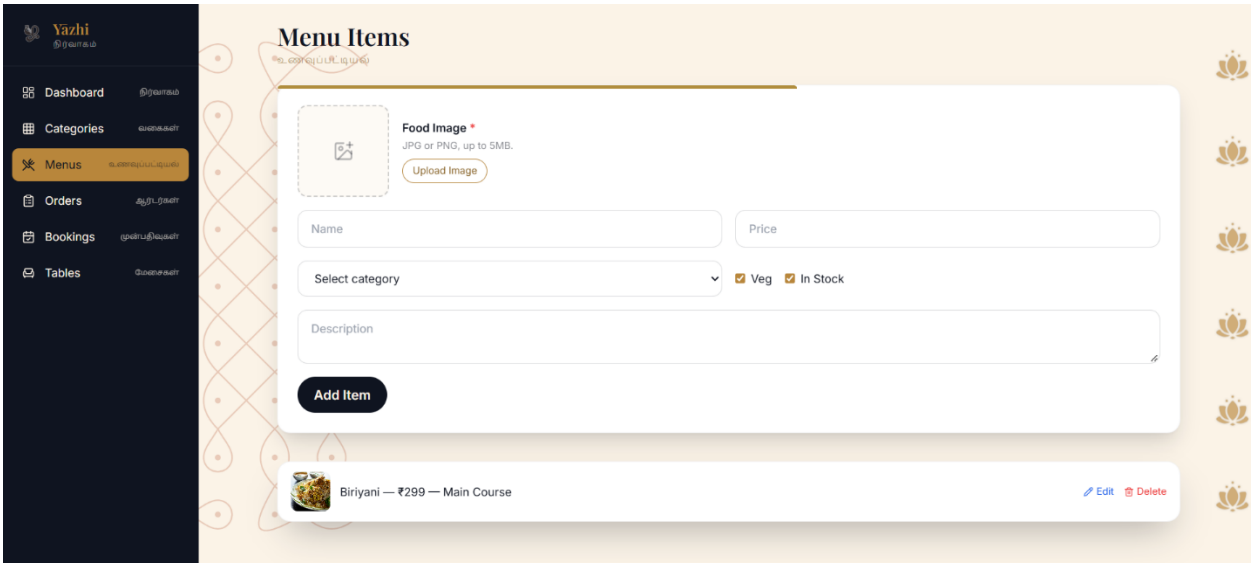

The customer-facing frontend of restaurant presents the restaurant through a home page featuring the brand, signature menu, restaurant highlights, atmosphere gallery, celebrations, and customer reviews. The About and Location sections provide information about the restaurant, its services, address, and opening hours. Customers can browse and search menu items, filter by category or vegetarian status, and view dish details. They can also sign in, enter reservation details, select a table through the interactive seating layout, and review their bookings. These pages use a consistent Tamil heritage design with kolam motifs and a brass, terracotta, and cream colour palette.

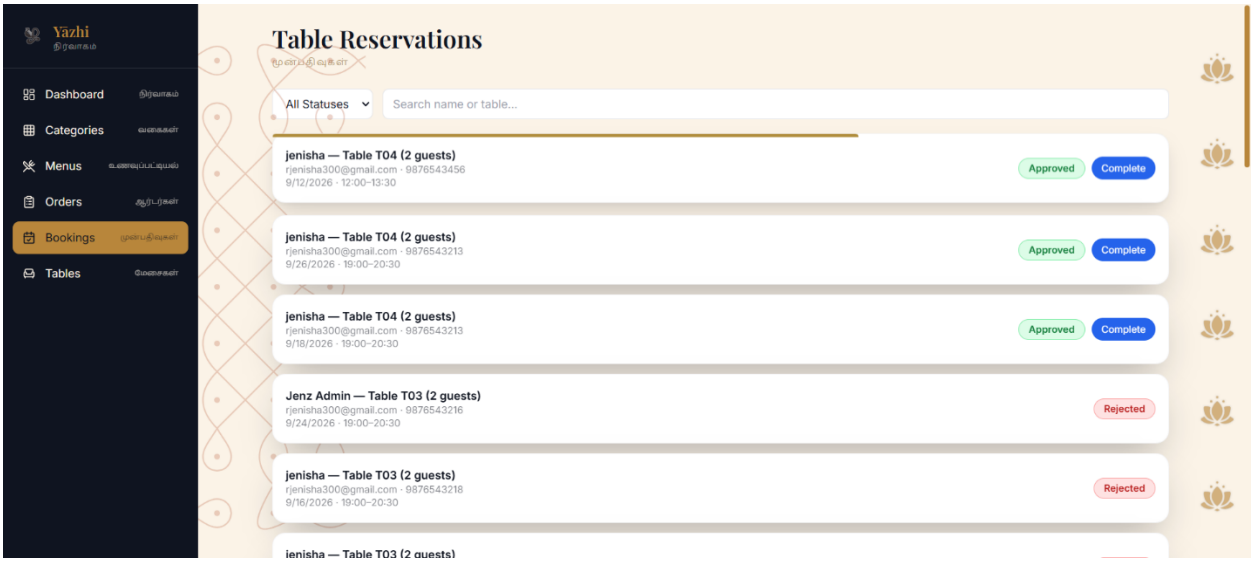
**ADMIN FRONTEND:**

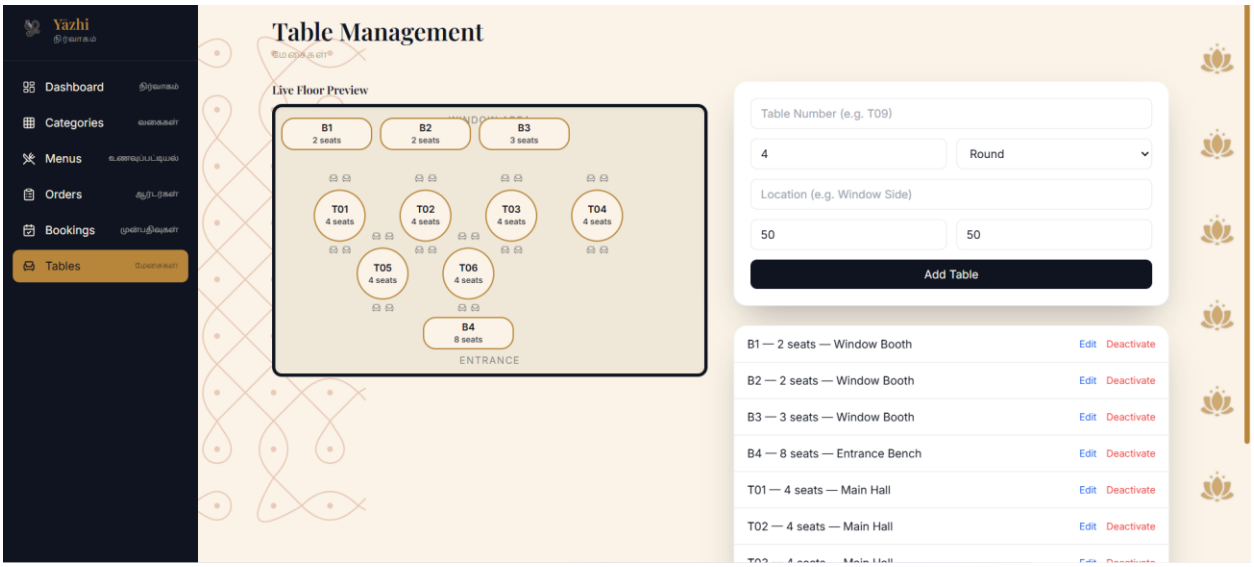
The admin panel provides a dashboard with summaries of orders, revenue, menu items, categories, and reservations, along with lists of recent orders and bookings. Administrators can manage menu categories and dishes, including item details, images, prices, and availability. The booking section supports searching and filtering reservations and lets administrators approve, reject, or complete bookings. The table management section displays a seating layout and allows administrators to add, edit, or deactivate tables and update their capacity, shape, location, and position.

ADMIN DASBOARD (FIGURE 8.4)




Yāzhi
Dashboard
Categories
Menus
Orders
Bookings
Tables
Table Reservations
All Statuses
Search name or table...
jenisha — Table T04 (2 guests)
rjenisha300@gmail.com · 9876543456
9/12/2026 · 12:00–13:30
Approved
Complete
jenisha — Table T04 (2 guests)
rjenisha300@gmail.com · 9876543213
9/26/2026 · 19:00–20:30
Approved
Complete
jenisha — Table T04 (2 guests)
rjenisha300@gmail.com · 9876543213
9/18/2026 · 19:00–20:30
Approved
Complete
Jenz Admin — Table T03 (2 guests)
rjenisha300@gmail.com · 9876543216
9/24/2026 · 19:00–20:30
Rejected
jenisha — Table T03 (2 guests)
rjenisha300@gmail.com · 9876543218
9/16/2026 · 19:00–20:30
Rejected


Yāzhi
Dashboard
Categories
Menus
Orders
Bookings
Tables
Table Management
Live Floor Preview
B1
2 seats
B2
2 seats
B3
3 seats
T01
4 seats
T02
4 seats
T03
4 seats
T04
4 seats
T05
4 seats
T06
4 seats
B4
8 seats
ENTRANCE
Table Number (e.g. T09)
4
Round
Location (e.g. Window Side)
50
50
Add Table
B1 — 2 seats — Window Booth
Edit
Deactivate
B2 — 2 seats — Window Booth
Edit
Deactivate
B3 — 3 seats — Window Booth
Edit
Deactivate
B4 — 8 seats — Entrance Bench
Edit
Deactivate
T01 — 4 seats — Main Hall
Edit
Deactivate
T02 — 4 seats — Main Hall
Edit
Deactivate

## Login Page :

The signup and login pages allow customers to create an account and access protected features such as ordering and table reservations.

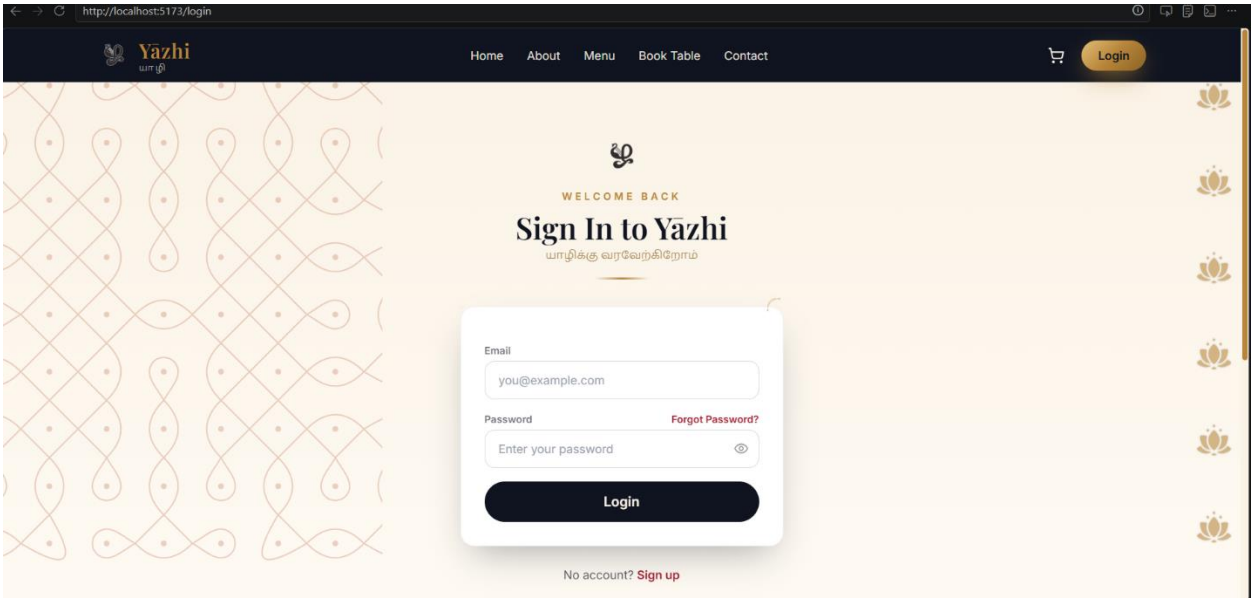



Figure 8.5 : Login Page

## 8.2 CODING

The folder is organized into separate frontend and backend folders rather than a single source file. The frontend contains React pages and reusable components, while the backend contains Express routes, controllers, middleware, and Mongoose models. The following excerpts illustrate key parts of the implementation.

## Module 1: Frontend Routing

React Router connects the customer pages and administrative pages.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/menu" element={<Menu />} />
  <Route path="/cart" element={<Cart />} />
```

```
  <Route path="/checkout" element={<Checkout />} />
  <Route path="/book-table" element={<BookTable />} />
  <Route path="/select-table" element={<SelectTable />} />
  <Route path="/my-bookings" element={<MyBookings />} />
  <Route path="/my-orders" element={<MyOrders />} />

  <Route path="/admin/login" element={<AdminLogin />} />
  <Route path="/admin" element={<AdminLayout />}>
    <Route path="dashboard" element={<Dashboard />} />
    <Route path="menus" element={<Menus />} />
    <Route path="orders" element={<Orders />} />
    <Route path="bookings" element={<Bookings />} />
    <Route path="tables" element={<TableManagement />} />
  </Route>
</Routes>
```

## Module 2: Frontend API Configuration

Axios is configured in the application context to communicate with the backend. The backend address can be supplied through the frontend environment configuration.

```
export const backendUrl =
import.meta.env.VITE_BACKEND_URL || "http://localhost:5000";

axios.defaults.baseURL = backendUrl;
axios.defaults.withCredentials = true;
```

## Module 3: Backend Setup and API Routes

The Express application connects to MongoDB, optionally configures Cloudinary, and mounts the API routes used by the frontend.

```
const app = express();
const PORT = process.env.PORT || 4000;

await connectDB();

if (process.env.CLOUDINARY_CLOUD_NAME) {
  await connectCloudinary();
}
```

```
app.use(express.json());
app.use(cookieParser());

app.use("/api/auth", authRouter);
app.use("/api/tables", tableRouter);
app.use("/api/booking", bookingRouter);
app.use("/api/menu", menuRouter);
app.use("/api/categories", categoryRouter);
app.use("/api/orders", orderRouter);
```

## Module 4: Authentication and Administrator Access

The authentication middleware checks for a JWT in the request cookie or Authorization header. The administrator middleware permits access only to accounts with the admin role.

```
export const protect = async (req, res, next) => {
  try {
    const token =
      req.cookies?.token ||
      (req.headers.authorization?.startsWith("Bearer")
        ? req.headers.authorization.split(" ")[1]
        : null);

    if (!token) {
      return res.status(401).json({
        success: false,
        message: "Not authorized, please login",
      });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select("-password");

    if (!user) {
      return res.status(401).json({
        success: false,
        message: "User not found",
      });
```

```
    }

    req.user = user;
    next();
  } catch {
    return res.status(401).json({
      success: false,
      message: "Invalid or expired token",
    });
  }
};

export const adminOnly = (req, res, next) => {
  if (req.user?.role !== "admin") {
    return res.status(403).json({
      success: false,
      message: "Admin access only",
    });
  }
  next();
};
```

## Module 5: Table Availability Check

The backend checks active tables with sufficient capacity and compares the requested time with existing Pending or Approved bookings. The overlap condition below is used to identify tables that are already booked during the requested interval.

```
const candidateTables = await Table.find({
  active: true,
  capacity: { $gte: Number(guests) },
}).sort({ tableNumber: 1 });

const activeBookings = await Booking.find({
  date: { $gte: dayStart, $lte: dayEnd },
  status: { $in: ["Pending", "Approved"] },
});

const blockedTableIds = new Set(
  activeBookings
    .filter((booking) => {
```

```
      const bookingStart = toMinutes(booking.startTime);
      const bookingEnd = toMinutes(booking.endTime);

      return requestedStartMinutes < bookingEnd &&
            bookingStart < requestedEndMinutes;
    })
    .map((booking) => booking.table.toString())
);
```

## Module 6: Order Creation

The order controller checks that the cart contains items, then associates the new order with the authenticated customer and stores it in MongoDB.

```
export const createOrder = async (req, res) => {
  try {
    const { items, amount, address, paymentMethod, note } = req.body;

    if (!items?.length) {
      return res.status(400).json({
        success: false,
        message: "Cart is empty",
      });
    }

    const order = await Order.create({
      user: req.user._id,
      items,
      amount,
      address,
      paymentMethod,
      note,
    });

    res.json({
      success: true,
      message: "Order placed successfully",
      order,
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message,
    });
  }};
```

# CHAPTER 9

# BIBLIOGRAPHY AND REFERENCES

## BOOK REFERENCES


1. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw Hill. Publisher information
2. Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps* (2nd ed.). O'Reilly Media. Publisher information
3. Casciaro, M., & Mammino, L. (2020). *Node.js Design Patterns* (3rd ed.). Packt Publishing. Publisher information
4. Bradshaw, S., Brazil, E., & Chodorow, K. (2019). *MongoDB: The Definitive Guide* (3rd ed.). O'Reilly Media. Publisher information


## WEB REFERENCES


1. React. https://react.dev/learn
2. Vite. . https://vite.dev/guide/
3. Tailwind CSS. . https://v3.tailwindcss.com/docs
4. OpenJS Foundation.. https://nodejs.org/docs/
5. Express.js. https://expressjs.com/en/4x/api/
6. Mongoose. . https://mongoosejs.com/docs/8.x/
7. MongoDB, Inc. . https://www.mongodb.com/docs/manual/
8. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). RFC Editor. https://www.rfc-editor.org/rfc/rfc7519
9. Cloudinary.(n.d.).*Image Upload Documentation.* https://cloudinary.com/documentation/image_upload_api_reference